Bancos de Dados Distribuídos

Gustavo Moretto Itikawa, Ingrid Louise Pereira Lohmann, Janaina Maria Cera da Silva, Ricardo de Oliveira Geraldo

Departamento de Informática
Universidade Estadual de Maringá (UEM) - Maringá, PR - Brasil

{ra90416@uem.br, ra117698@uem.br, ra115832@uem.br, ra124583@uem.br }

Resumo. *Este artigo explora os fundamentos, as arquiteturas e os desafios dos Sistemas de Gerenciamento de Banco de Dados Distribuídos (SGBDDs), cruciais para a escalabilidade e a confiabilidade na era digital. O estudo indica que a transparência, essencial ao usuário, é assegurada pelas estratégias de projeto: Fragmentação, Replicação e Alocação. Contudo, o principal desafio é o Controle de Concorrência, que exige protocolos como o Two-Phase Commit (2PC) e o gerenciamento do deadlock distribuído. Conclui-se que o gerenciamento de falhas e o design de arquiteturas resilientes são focos centrais para a evolução contínua dos SGBDDs.*

1. Introdução

A crescente demanda por sistemas capazes de lidar com grandes volumes de dados, distribuídos geograficamente e acessados por múltiplos usuários, impulsionou o desenvolvimento dos Sistemas de Gerenciamento de Banco de Dados Distribuídos (SGBDDs). Diferentemente de bancos de dados centralizados, esses sistemas integram múltiplos bancos logicamente relacionados, mas fisicamente espalhados em diversos nós da rede, preservando para o usuário a ilusão de uma base de dados unificada (Özsu; Valduriez, 2020). Essa transparência é um dos pilares do modelo distribuído e se manifesta em aspectos como localização, replicação e fragmentação dos dados.

A adoção de uma arquitetura distribuída também atende a necessidades práticas de organizações modernas, como maior disponibilidade, escalabilidade e desempenho. A proximidade entre dados e usuários reduz a latência, enquanto a replicação aumenta a tolerância a falhas. No entanto, esses benefícios vêm acompanhados de desafios técnicos significativos. Entre eles, destacam-se a manutenção da consistência global das transações, a coordenação entre múltiplos nós, o tratamento de falhas e o alto custo de comunicação inerente ao processamento distribuído.

Diante desse cenário, compreender o funcionamento interno dos SGBDDs é essencial para avaliar suas vantagens, limitações e aplicações. Este artigo apresenta os conceitos fundamentais do modelo distribuído, as principais estratégias de projeto e as técnicas que permitem garantir integridade, desempenho e resiliência em ambientes descentralizados.

2. Desenvolvimento

Primeiramente, foi realizada uma pesquisa exploratória para identificação e definição dos conceitos fundamentais do tema "Bancos de Dados Distribuídos", o que permitiu o

mapeamento dos principais desafios e arquiteturas a serem abordados. Os tópicos centrais incluíram: Conceitos de Transparência e Consistência, Estratégias de Projeto (Fragmentação, Replicação e Alocação) e o Controle de Concorrência.

O processo de pesquisa bibliográfica foi dividido em três fases:

1. **Levantamento de Fontes Fundamentais:** Priorizamos a consulta a livros didáticos clássicos e atuais, como *Principles of Distributed Database Systems* (Özsu; Valduriez, 2020) e *Fundamentals of Database Systems* (Elmasri; Navathe, 2016), que fornecem a base teórica e conceitual da área.
2. **Busca e Seleção de Artigos Científicos:** Em seguida, foram consultadas bases de dados de renome na área de Computação, como Google Acadêmico, ACM Digital Library e Scielo. Essa busca visou encontrar artigos científicos que detalhassem os desafios técnicos e inovações, como as técnicas de revisão sistemática (Kituta et al., 2019) e o controle de concorrência (Bernstein; Goodman, 1981).
3. **Análise e Síntese:** Por fim, o material foi analisado e sintetizado, garantindo que os dados coletados fossem pertinentes às seções subsequentes, detalhando os fundamentos, as arquiteturas e os mecanismos internos de um SGBDD.

2.1 Conceitos Fundamentais do SGBDD

A característica definidora de um Sistema de Gerenciamento de Banco de Dados Distribuído (SGBDD) é a transparência (Özsu; Valduriez, 2020). Este conceito garante que, apesar dos dados estarem logicamente inter-relacionados e fisicamente dispersos em diversos nós de uma rede, o sistema se apresente como uma única entidade lógica para o usuário (Kituta et al., 2019). A transparência, assim, alivia o usuário da complexidade de saber onde os dados estão armazenados e como são acessados (Özsu; Valduriez, 2020).

O projeto de um SGBDD é determinado por três estratégias principais (Özsu; Valduriez, 2020; Kituta et al., 2019):

- **Fragmentação:** É o processo de dividir uma relação (tabela) em subconjuntos menores, permitindo que as partes da tabela sejam armazenadas em locais diferentes. A fragmentação pode ser Horizontal (dividir em linhas, baseada em critérios lógicos) ou Vertical (dividir em colunas).
- **Replicação:** Consiste em manter cópias exatas dos dados (fragmentos) em diferentes nós da rede (Kituta et al., 2019). A replicação é crucial para aumentar a disponibilidade (o sistema continua operando mesmo que um nó falhe) e o desempenho (consultas podem ser executadas localmente).
- **Alocação:** É a decisão sobre qual nó da rede armazenará cada fragmento (e suas réplicas). A alocação busca otimizar o acesso aos dados e minimizar os custos de comunicação.

2.2 Ilustração da Arquitetura

A complexidade e a interconexão de um SGBDD podem ser melhor visualizadas por meio de uma representação gráfica.

A Figura 1 ilustra a arquitetura de um Sistema de Gerenciamento de Banco de Dados Distribuído (SGBDD) Homogêneo. Ela demonstra a interligação de múltiplos SGBDs idênticos, que se comunicam através de uma rede, gerenciados por um módulo de Transação e um módulo de Processamento de Consultas global.

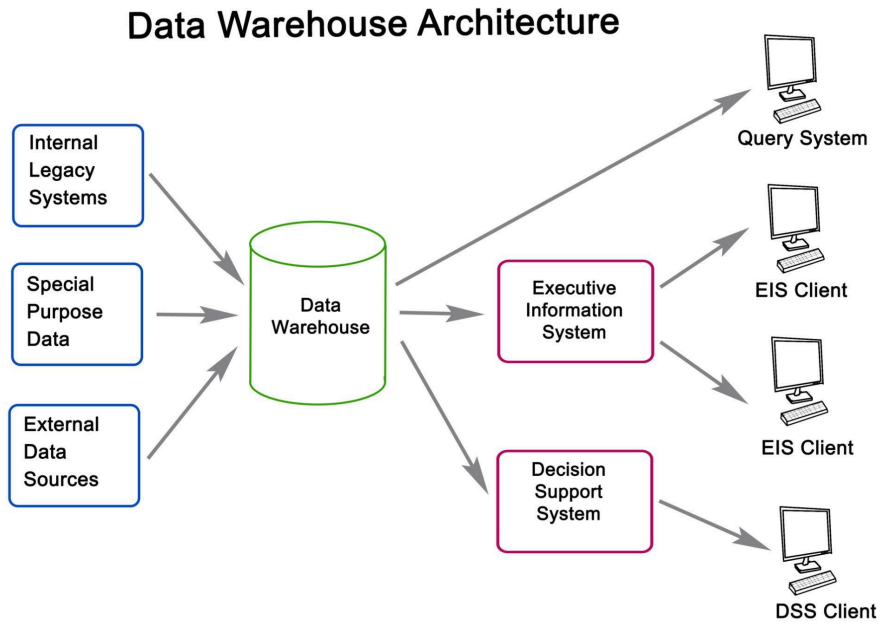

Figura 1. Arquitetura de um Sistema de Gerenciamento de Banco de Dados Distribuídos (Adaptado de Özsu; Valduriez, 2020)

A figura destaca a separação entre o esquema global, percebido pelo usuário, e os esquemas de fragmentação locais em cada nó (SGBD). Essa arquitetura reforça o conceito central de transparência da distribuição, onde o sistema se comporta como um único SGBD lógico para o usuário final.

2.3 Controle de Concorrência e Gerenciamento de Transações

O Controle de Concorrência é, sem dúvida, o problema técnico mais complexo enfrentado pelos SGBDDs (Bernstein; Goodman, 1981). Enquanto em sistemas centralizados todas as operações são gerenciadas por um único agendador (scheduler), nos sistemas distribuídos, uma transação distribuída é dividida em sub transações executadas em diferentes nós da rede (Özsu; Valduriez, 2020).

Para garantir a integridade dos dados em um ambiente disperso, as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade) devem ser mantidas globalmente (Elmasri; Navathe, 2016). Dois protocolos são cruciais para isso:

2.3.1 Atomicidade (Two-Phase Commit - 2PC)

Para que uma transação distribuída seja atômica (ou tudo acontece, ou nada acontece), é necessário o protocolo de Two-Phase Commit (2PC) (Özsu; Valduriez, 2020). Este protocolo garante que todos os nós envolvidos na transação cheguem a um consenso global:

- **Fase 1 (Voto):** O Coordenador (geralmente o nó onde a transação se originou) solicita a todos os participantes um voto sobre a preparação da transação. Se todos os participantes puderem confirmar, eles votam PREPARAR; se houver falha, votam ABORTAR.

- **Fase 2 (Decisão):** Se todos os votos forem PREPARAR, o Coordenador envia a decisão COMMIT (confirmar) a todos os participantes. Caso contrário, envia ABORT (abortar).

2.3.2 Isolamento (Distributed Locking)

O isolamento, que lida com o acesso simultâneo aos dados, é implementado por técnicas de bloqueio distribuído, como o Distributed Two-Phase Locking (2PL) (Bernstein; Goodman, 1981). O 2PL distribui a responsabilidade de bloqueio entre os vários nós, garantindo que a execução concorrente seja equivalente a uma execução serial. A principal complexidade deste método reside em evitar deadlocks (impasse) e em otimizar a comunicação para obter e liberar bloqueios entre os nós distribuídos (Bernstein; Goodman, 1981).

2.3.3 O Problema do Deadlock Distribuído

Um *deadlock* (impasse) ocorre quando um conjunto de transações está preso em um ciclo de espera, onde cada transação aguarda a liberação de um recurso detido por outra transação no ciclo (Özsu; Valduriez, 2020). Em sistemas distribuídos, esse problema é exacerbado, pois a dependência de espera se estende por diferentes nós, e nenhum nó individual possui o grafo completo de espera da transação (Bernstein; Goodman, 1981).

O desafio reside na **detecção** de ciclos de espera. As soluções mais comuns envolvem:

- **Detecção Global:** Requer que os nós periodicamente troquem informações sobre seus grafos de espera locais. Um nó Coordenador monta e analisa o grafo global para identificar ciclos, o que resulta em alta sobrecarga de comunicação (Özsu; Valduriez, 2020).
- **Prevenção:** Técnicas como o *wait-die* (espera-morre) ou *wound-wait* (fere-espera) evitam deadlocks ao impor regras estritas baseadas na idade da transação. Essas regras forçam o aborto de uma transação para quebrar o ciclo antes que ele se forme (Elmasri; Navathe, 2016).

A escolha da estratégia de gerenciamento de *deadlock* é crucial, pois a detecção distribuída aumenta a sobrecarga de comunicação, impactando diretamente o desempenho geral do SGBDD (Bernstein; Goodman, 1981).

2.4 Processamento e Otimização de Consultas Distribuídas

O processamento de consultas em um SGBDD é significativamente mais complexo do que em um sistema centralizado, devido à necessidade de mover dados entre múltiplos nós conectados por uma rede (Özsu; Valduriez, 2020). O principal objetivo da otimização de consultas distribuídas é **minimizar o custo da comunicação de dados**, que geralmente é a métrica mais dispendiosa do sistema (Elmasri; Navathe, 2016).

O processo de consulta é dividido em três fases principais:

1. **Decomposição da Consulta:** A consulta global é traduzida em operações relacionais (como seleção, projeção e junção) e simplificada. Esta fase é independente da distribuição dos dados.
2. **Localização dos Dados:** Otimiza a consulta inicial, convertendo-a em consultas de fragmentos (frag-queries) que operam nos fragmentos físicos. Se houver redundância (replicação), o otimizador deve decidir qual réplica é a mais eficiente para acessar (Özsu; Valduriez, 2020).
3. **Otimização Global e Execução:** Otimiza o plano de execução, levando em conta o custo da comunicação de dados entre os nós. O objetivo é escolher uma sequência de junções e movimentos de dados que reduza o tráfego de rede (Kituta et al., 2019). Técnicas como a semijunção (*semi-join*) são amplamente utilizadas para reduzir o tamanho das tabelas a serem transferidas, enviando apenas os valores necessários para a junção, diminuindo drasticamente o custo de comunicação (Elmasri; Navathe, 2016).

2.5 Arquiteturas e Heterogeneidade

A arquitetura de um SGBDD descreve como os sistemas de gerenciamento de banco de dados (SGBDs) individuais estão interligados. Os sistemas são geralmente classificados com base na uniformidade dos SGBDs nos diferentes nós (Özsu; Valduriez, 2020).

- **Sistemas Homogêneos:** Todos os sites usam o mesmo software de SGBD (o mesmo fornecedor, a mesma versão). O esquema global é idêntico em todos os locais, o que simplifica o gerenciamento de transações e a otimização de consultas.
- **Sistemas Heterogêneos:** Os sites utilizam diferentes SGBDs (por exemplo, Oracle em um nó, PostgreSQL em outro) e podem usar diferentes modelos de dados ou esquemas. Esta arquitetura reflete a realidade da maioria das grandes organizações, mas introduz complexidades significativas, exigindo um *middleware* para traduzir comandos e garantir a transparência da linguagem e dos dados (Özsu; Valduriez, 2020).

A tendência arquitetônica moderna utiliza modelos como Cliente-Servidor e, mais recentemente, a arquitetura de bancos de dados distribuídos nativos em nuvem (*cloud-native*), onde a fragmentação e a replicação são gerenciadas automaticamente pela plataforma de *cloud computing* (Kituta et al., 2019).

2.6 Gerenciamento de Falhas e Recuperação de Dados

Em um ambiente distribuído, a probabilidade de falha é significativamente maior, pois o sistema deve lidar não apenas com falhas de nó (processador, memória), mas também com falhas de comunicação e particionamento da rede (Özsu; Valduriez, 2020). Um SGBDD robusto deve garantir as propriedades ACID, mesmo diante desses eventos.

O protocolo Two-Phase Commit (2PC), já introduzido, é o mecanismo primário para manter a atomicidade distribuída em caso de falha de um nó participante ou da rede. No entanto, o 2PC tem uma limitação: se o coordenador falhar *depois* de ter enviado o voto PREPARAR (Fase 1), mas *antes* de ter enviado a decisão COMMIT (Fase 2), os participantes que votaram PREPARAR podem ficar em um estado bloqueado (*blocking state*) (Elmasri; Navathe, 2016).

Para garantir a Durabilidade e permitir a recuperação, os SGBDDs utilizam:

- **Logs de Transação Distribuídos:** Cada nó mantém um log local das transações que participou. Estes logs são essenciais para que, após a reinicialização de um nó, o sistema possa executar operações de *undo* (desfazer transações incompletas) ou *redo* (refazer transações confirmadas, mas não totalmente persistidas) de forma consistente com os demais nós.

- **Checkpoints:** Pontos regulares de sincronização no sistema, que reduzem a quantidade de trabalho necessária durante a recuperação, limitando a busca nos logs a um intervalo específico (Özsu; Valduriez, 2020).

A replicação dos dados (discutida em 2.1) é o principal fator que garante a confiabilidade global do sistema, pois a falha de um nó não resulta na indisponibilidade dos dados (Elmasri; Navathe, 2016).

3. Conclusão

O objetivo deste artigo foi explorar os fundamentos, as arquiteturas e os desafios técnicos inerentes aos Sistemas de Gerenciamento de Banco de Dados Distribuídos (SGBDDs). Conclui-se que os SGBDDs são essenciais para atender às demandas de confiabilidade e escalabilidade da era moderna.

Os resultados da pesquisa demonstram que a eficácia de um SGBDD depende da correta implementação das estratégias de projeto – Fragmentação, Replicação e Alocação – que, em conjunto, garantem a transparência da distribuição para o usuário final (Kituta et al., 2019; Özsu; Valduriez, 2020). Essa transparência é suportada por arquiteturas que variam de modelos homogêneos e simples a complexos sistemas heterogêneos.

A natureza distribuída impõe desafios significativos no gerenciamento de transações, exigindo protocolos robustos. O Controle de Concorrência é mantido pelo bloqueio distribuído (2PL) e pela atomicidade do Two-Phase Commit (2PC). No entanto, o 2PC não elimina o risco de estados bloqueados, e o problema do deadlock

distribuído adiciona uma sobrecarga considerável ao sistema devido à complexidade de sua detecção global (Bernstein; Goodman, 1981). Além disso, o Processamento de Consultas Distribuídas é dominado pela necessidade de minimizar o custo de comunicação de dados (Elmasri; Navathe, 2016).

Por fim, o Gerenciamento de Falhas é uma preocupação permanente, onde logs de transação e *checkpoints* se unem à replicação de dados para garantir a durabilidade e a resiliência do sistema. A otimização desses protocolos e a adaptação a ambientes *cloud-native* continuam sendo os eixos centrais de pesquisa e desenvolvimento em SGBDDs.

4. Referências

BERNSTEIN, P. A.; GOODMAN, N. Concurrency Control in Distributed Database Systems. ACM Computing Surveys, v. 13, n. 2, p. 185–221, 1981. DOI: 10.1145/356842.356846.

ELMASRI, R.; NAVATHE, S. Fundamentals of Database Systems. 7. ed. Londres: Pearson, 2016.

KITUTA, E. et al. A SYSTEMATIC REVIEW ON DISTRIBUTED DATABASES SYSTEMS AND THEIR TECHNIQUES. Journal of Theoretical and Applied Information Technology, v. 97, n. 1, p. 198-208, 2019. Disponível em: http://www.jatit.org/volumes/Vol97No1/21Vol97No1.pdf. Acesso em: 21 nov. 2025.

ÖZSU, M. T.; VALDURIEZ, P. Principles of Distributed Database Systems. 4. ed. Nova York: Springer, 2020.